Environnements de travail

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. PyCharm **3**

A. Présentation de PyCharm 3

B. Installation de PyCharm 5

C. Création d'un projet et installation des packages....5

D. Exercice : Quiz 7

III. VSCode **8**

A. Présentation de VSCode 8

B. Installation de VSCode....9

C. Exercice : Quiz 10

IV. Essentiel **11**

V. Auto-évaluation **12**

A. Exercice....12

Solutions des exercices **12**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 60 minutes

Environnement de travail : PC ou Mac avec 8 Go de ram minimum

Contexte

Avant de plonger dans l'apprentissage du langage Python, il est essentiel de choisir et de maîtriser un environnement de développement adapté. Ce cours a pour objectif de vous familiariser avec deux outils majeurs utilisés pour programmer en Python : PyCharm et VSCode.

Nous commencerons par PyCharm, un IDE puissant conçu spécialement pour Python, offrant des fonctionnalités avancées comme le débogage, la gestion des environnements virtuels et l'intégration native avec des frameworks populaires. Ensuite, nous explorerons Visual Studio Code (VSCode), un éditeur de code léger et flexible, qui grâce à ses nombreuses extensions, permet de configurer un environnement Python efficace et personnalisé.

À travers ce cours, vous apprendrez à installer et configurer les IDE PyCharm et VSCode, puis à gérer des environnements virtuels pour organiser vos projets, mais aussi à installer et utiliser des packages Python. Découvrons ensemble ce nouvel environnement !

Objectifs pédagogiques : se familiariser avec deux outils majeurs utilisés pour programmer en Python : PyCharm et VSCode.

Logos officiels dePyCharm et VSCode

Source : JETBRAINS[1] et Visual studio[2]

II. PyCharm

A. Présentation de PyCharm

PyCharm est un environnement de développement intégré (IDE) spécialement conçu pour Python, développé par JetBrains. Il simplifie l'écriture, le débogage et la gestion des projets Python, et est apprécié par les développeurs professionnels, les entreprises et les chercheurs en data science.

1. https://www.jetbrains.com/fr-fr/pycharm/
2. https://code.visualstudio.com/

PyCharm est proposé en deux éditions principales. PyCharm Community et PyCharm Professional.

La version **Community**, entièrement gratuite, est idéale pour les développeurs débutants ou ceux qui travaillent sur des projets Python classiques. Elle inclut les fonctionnalités essentielles telles que l'auto-complétion intelligente, le débogueur graphique, la gestion des environnements virtuels et l'intégration avec Git.

En revanche, la version **Professional**, payante, est destinée aux développeurs avancés et aux entreprises. Elle offre des fonctionnalités supplémentaires comme le support des frameworks web (Django, Flask), une meilleure prise en charge des bases de données, ainsi que des outils avancés pour l'analyse de code et le développement collaboratif.

Pour les étudiants et les enseignants, JetBrains propose également une licence gratuite pour PyCharm Professional, sur simple demande.

Avantages de PyCharm

PyCharm est largement reconnu comme l'un des environnements de développement intégrés (IDE) les plus puissants pour le langage Python. Voici un aperçu détaillé des avantages qu'il offre :

- **Éditeur de code intelligent**

  PyCharm offre un éditeur de code intelligent qui propose des **suggestions de code contextuelles**, permettant aux développeurs d'écrire du code plus rapidement et avec moins d'erreurs. Grâce à l'analyse de code en temps réel, PyCharm **détecte automatiquement les erreurs de syntaxe, les avertissements, et les problèmes potentiels**, permettant de les corriger immédiatement. De plus, les outils de refactorisation facilitent les modifications de code, comme le renommage de variables ou l'extraction de méthodes, de manière sécurisée et efficace.

- **Débogage et tests**

  PyCharm se distingue par son débogueur graphique intégré, qui permet d'exécuter le code pas à pas, d'inspecter les variables en temps réel et de repérer facilement les erreurs, sans se limiter aux simples instructions print(). Grâce aux points d'arrêt, à l'évaluation d'expressions et à l'inspection des variables, il offre un contrôle précis sur l'exécution du programme. De plus, PyCharm facilite l'écriture et l'exécution de tests unitaires en s'intégrant directement avec des frameworks comme **pytest** et **unittest**, permettant ainsi aux développeurs de garantir la fiabilité de leur code sans quitter l'IDE.

- **Gestion de versions**

  PyCharm s'intègre parfaitement avec Git et GitHub, offrant une interface intuitive pour gérer le contrôle de version. Les développeurs peuvent committer, pousser et fusionner du code directement depuis l'IDE, ce qui simplifie la gestion des projets collaboratifs.

- **Environnement de développement complet**

  PyCharm propose un environnement de développement complet, avec un support natif pour de nombreux frameworks Python tels que **Django, Flask, et Pyramid**, simplifiant ainsi le développement web. Pour les projets de data science, PyCharm s'intègre également avec des outils comme **Jupyter et Anaconda**, ainsi qu'avec des bibliothèques scientifiques, offrant ainsi un environnement idéal pour le développement scientifique.

En plus de ces avantages, PyCharm offre une grande flexibilité grâce à sa capacité de personnalisation. Les développeurs peuvent adapter l'interface utilisateur et les fonctionnalités de l'IDE à leurs préférences personnelles, en utilisant une vaste bibliothèque de plugins. De plus, PyCharm bénéficie d'une communauté active et d'un support régulier de la part de JetBrains, garantissant des mises à jour fréquentes et un accès à de nombreuses ressources en ligne pour obtenir de l'aide et des conseils. Cette combinaison de personnalisation et de soutien communautaire fait de PyCharm un choix privilégié pour de nombreux développeurs Python.

Tous droits réservés à STUDI - Reproduction interdite

B. Installation de PyCharm

L'installation de PyCharm est une étape simple et rapide, compatible avec Windows, macOS et Linux. Voici comment procéder :

Télécharger PyCharm compatible avec votre système d'exploitation soit en version Community(gratuit) ou Professional (payant) sur le site officiel du développeur JetBrains : **JETBRAINS**[3]

Sous Windows :

Une fois le fichier téléchargé, double-cliquez dessus pour lancer l'installateur.L'installateur vous guidera à travers le processus d'installation. Vous pouvez accepter les paramètres par défaut ou les personnaliser selon vos préférences.

Après l'installation, vous pouvez lancer PyCharm directement depuis le menu Démarrer ou en utilisant le raccourci sur le bureau.

Sous Mac :

Double-cliquez sur le fichier .dmg téléchargé pour monter l'image disque. Dans la fenêtre qui s'ouvre, faites glisser l'icône de PyCharm dans le dossier Applications pour l'installer.

Vous pouvez maintenant lancer PyCharm depuis le dossier Applications.

Sous Linux :

Ouvrez un terminal et extrayez le fichier .tar.gz téléchargé en utilisant la commande `tar -xzf pycharm-*.tar.gz`.

Déplacez le dossier extrait vers le répertoire souhaité, par exemple `/opt/pycharm`.

Accédez au répertoire d'installation et exécutez le script `pycharm.sh` pour lancer PyCharm.

Lors du premier lancement de PyCharm, vous serez guidé à travers une configuration initiale qui inclut l'importation de paramètres précédents (si disponibles), le choix du thème d'interface (clair ou sombre), et la configuration de l'interpréteur Python, avec l'option d'utiliser un interpréteur existant ou d'en télécharger un via PyCharm.

C. Création d'un projet et installation des packages

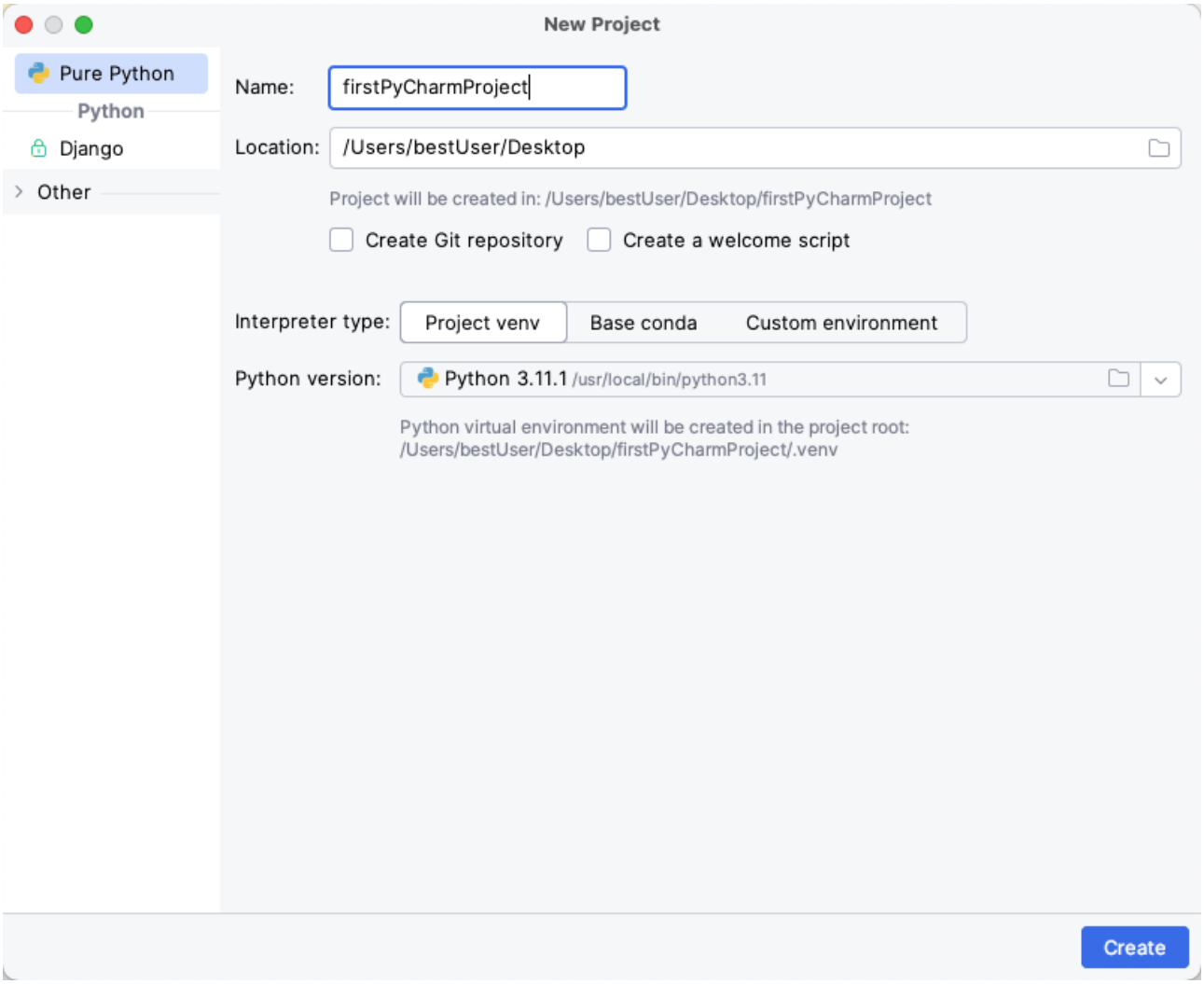
Créer le projet

Pour créer tout d'abord un projet :

- Lancez PyCharm
- Cliquez sur *new project*
- Modifiez le nom du projet en *firstPyCharmProject* par exemple
- Choisissez Project *venv*
- Sélectionnez la version de Python voulue
- Cliquez sur *Create*

3. https://www.jetbrains.com/pycharm/download/

Tous droits réservés à STUDI - Reproduction interdite



Remarque **Pourquoi choisir *venv* dans PyCharm ?**

Lors de la création d'un projet Python dans PyCharm, il est recommandé d'utiliser un environnement virtuel (*venv*). Un *venv* permet d'isoler chaque projet avec ses propres bibliothèques, sans modifier votre installation Python globale. Cette approche évite les conflits entre projets et simplifie grandement la gestion des dépendances.

Ainsi débuter avec *venv* est une excellente solution à moins que ne travaillez déjà dans un environnement virtuel comme *Docker* ou dans un projet global nécessitant un accès constant aux mêmes bibliothèques sur tout le système.

Installer un package

Un package est un ensemble de modules Python qui ajoutent des fonctionnalités supplémentaires. Les packages sont gérés par *PyPI* (Python Package Index) et installés via *pip*, le gestionnaire de paquets de Python.

Parmi les plus populaires citons *NumPy* (Calculs mathématiques et tableaux numèriques), *Pandas* (Analyse et manipulation de données), *Flask* (Développement d'applications web) et *Requests* (Requêtes HTTP pour interagir avec des API.

Installons par l'interface graphique NumPy.

Tous droits réservés à STUDI - Reproduction interdite

- Allez dans **Settings** (dans le menu PyCharm sur Mac et le menu file sur Windows et Linux)
- Allez dans **Project: firstPyCharmProject**
- Puis **Python Interpreter**
- Cliquez sur **+** et taper **NumPy** dans le champ de recherche
- Cliquez sur **Install Package**

Il est possible d'installer le package via le terminal :

```
pip3 install numpy
```

Exemple

Créons un petit script pour tester NumPy :

- Dans **firstPyCharmProject**, faites un clic droit sur le dossier du projet.
- Sélectionnez **New**, **Python File** et nommez-le **test_numpy.py**

Ajoutez le code suivant :

```
import numpy as np

# Création d'un tableau Numpy
tableau = np.array([1,2,3,4,5])
print("Tableau Numpy :", tableau)
```

En exécutant ce script (clic droit → **Run 'test_numpy'**), vous devriez obtenir :

```
Tableau NumPy : [1 2 3 4 5]
```

Voilà vous avez fait vos premier pas avec PyCharm en utilisant le package NumPy. Vous êtes prêt pour commencer à coder !

D. Exercice : Quiz

[solution n°1 p.13]

Question 1

PyCharm est développé par Microsoft.

- ○ Vrai
- ○ Faux

Question 2

Quelle édition de PyCharm propose des fonctionnalités avancées comme le support des frameworks web (Django, Flask) ?

- ○ PyCharm Community
- ○ PyCharm Professional

Question 3

Quel est l'avantage principal d'utiliser un environnement virtuel (venv) dans PyCharm ?

- ○ Améliorer les performances du système
- ○ Isoler les dépendances de chaque projet
- ○ Rendre le code Python plus rapide

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Pourquoi est-il recommandé d'utiliser un environnement virtuel (venv) dans PyCharm ?

- ○ Pour améliorer les performances du processeur
- ○ Pour isoler les dépendances de chaque projet et éviter les conflits entre bibliothèques
- ○ Pour installer automatiquement toutes les bibliothèques disponibles

Question 5

Lors de la création d'un projet dans PyCharm, quelle étape ne fait pas partie du processus décrit dans le cours ?

- ○ Sélectionner la version de Python
- ○ Choisir un thème d'interface
- ○ Donner un nom au projet

III. VSCode

A. Présentation de VSCode

Visual Studio Code (VSCode) est un éditeur de texte gratuit, léger et polyvalent, développé par Microsoft. Très apprécié pour sa flexibilité, VSCode supporte un grand nombre de langages de programmation, dont Python. Grâce à ses nombreuses extensions et à sa simplicité d'utilisation, il est adopté aussi bien par les débutants que par les développeurs expérimentés.

Remarque **Versions disponibles de VSCode**

VSCode est un éditeur entièrement gratuit et open source, disponible pour Windows, macOS et Linux. Contrairement à PyCharm, il n'existe qu'une seule version gratuite, complétée par une large gamme d'extensions à installer selon vos besoins. Il est léger, rapide, et personnalisable grâce à son vaste écosystème de plugins.

Avantages de VSCode

Voici les principaux avantages qui font de VSCode un choix très populaire parmi les développeurs Python :

- **Léger et performant**

  VSCode est léger, rapide à lancer et consomme peu de ressources système, ce qui le rend particulièrement adapté aux systèmes moins puissants ou aux utilisateurs préférant une expérience rapide et fluide.

- **Extensions puissantes et polyvalentes**

  Grâce à l'extension officielle Python, VSCode dispose d'une auto-complétion intelligente (Pylance), un débogueur graphique puissant, et la gestion directe des environnements virtuels. Des extensions supplémentaires comme GitLens améliorent l'intégration avec Git/GitHub.

- **Débogage et tests simplifiés**

  VSCode intègre un débogueur graphique complet permettant d'inspecter les variables en temps réel, d'ajouter des points d'arrêt et de contrôler précisément l'exécution de vos scripts Python. Il facilite aussi l'utilisation des frameworks de tests tels que pytest ou unittest directement dans l'éditeur.

- **Flexibilité et personnalisation**

  Grâce à son écosystème d'extensions riche et varié, vous pouvez personnaliser VSCode en fonction de vos besoins spécifiques. Cette flexibilité rend VSCode idéal pour travailler simultanément avec plusieurs langages et outils différents.

- **Communauté active et support continu**

Tous droits réservés à STUDI - Reproduction interdite

Développé par Microsoft, VSCode bénéficie d'une très large communauté active, assurant des mises à jour fréquentes, de nouvelles fonctionnalités, et une documentation complète en ligne, idéale pour apprendre et résoudre rapidement vos problèmes.

B. Installation de VSCode

L'installation de l'éditeur VSCode

Télécharger VSCode adapté à votre système d'exploitation depuis le site officiel de Microsoft : Visual studio[4]

Sous Windows : double-cliquez sur l'installateur `.exe`, puis suivez les instructions en conservant les options par défaut recommandées.

Sous macOS : double-cliquez sur le fichier `.zip` ou `.dmg`, puis glissez Visual Studio Code dans le dossier Applications.

Sous Linux : téléchargez le fichier `.deb` (Debian/Ubuntu) ou `.rpm` selon votre distribution, puis installez-le avec le gestionnaire de paquets ou en ligne de commande.

- Pour Debian/Ubuntu :

```
sudo dpkg -i vscode.deb
sudo apt-get install -f
```

- Pour Fedora :

```
sudo rpm -i vscode.rpm
```

Lors du premier lancement, vous pouvez installer l'extension Python officielle en cliquant sur l'icône des extensions et en recherchant « *Python* ».

Complément **Les extensions recommandées**

Voici les extensions essentielles à installer dans VSCode pour une expérience optimale en Python :

Extension	Description
Python	**Extension officielle par Microsoft pour l'auto-complétion avancée, débogage intégré et gestion des environnements virtuels.**
Pylance	**Analyse statique avancée du code Python, offrant des suggestions intelligentes et précises.**
GitLens	**Gestion simplifiée de Git, visualisation claire des historiques des commits et amélioration du travail collaboratif.**
Code Runner	**Permet d'exécuter rapidement des scripts Python directement depuis l'éditeur.**
Black Formatter	**Formate automatiquement votre code selon le style Python (PEP8), facilitant la lecture et la cohérence du code.**

4. https://code.visualstudio.com/

Tous droits réservés à STUDI - Reproduction interdite

Installation des extensions

Cliquez sur l'icône des extensions à gauche (ou appuyez sur **Ctrl + Shift +X** sous Windows/Linux ou **Cmd + Shift + X** sous macOS).

Tapez le nom de l'extension dans la barre de recherche.

Cliquez sur « ***Install*** » pour ajouter l'extension à VSCode.

Bien que les extensions soient très utiles, veillez à ne pas en installer excessivement pour éviter de surcharger VSCode et conserver ses performances optimales.

Remarque **Pourquoi créer un environnement virtuel dans VSCode ?**

Comme dans PyCharm, il est fortement conseillé d'utiliser un environnement virtuel (`venv`) avec VSCode. Cela vous permet d'isoler les dépendances du projet, d'éviter les conflits entre bibliothèques et de faciliter la gestion des dépendances pour chaque projet.

Pour créer un environnement virtuel directement depuis VSCode :

Tapez dans le terminal intégré à VSCode :

```
python3 -m venv .venv
```

Activez-le en cliquant en bas à droite sur l'interpréteur Python affiché et sélectionnez celui du dossier `.venv`.

Installer un package dans VSCode

Installez le package requests depuis VSCode, qui permet de gérer simplement les requêtes HTTP :

Tapez la commande suivante :

```
pip3 install requests
```

Exemple **Tester Requests dans VSCode**

Ajoutez le code suivant dans votre fichier `main.py` :

```
import requests

response = requests.get('https://api.github.com')
print("Statut de la réponse HTTP :", response.status_code)
```

Exécutez le script en cliquant sur le bouton en haut à droite (run Python File) de VSCode ou en appuyant sur F5.

Si le résultat affiché est :

```
Statut de la réponse HTTP : 200
```

Bravo, votre premier projet Python avec Requests dans VSCode est fonctionnel !

C. Exercice : Quiz

[solution n°2 p.14]

Question 1

VSCode est développé par JetBrains.

- ○ Vrai
- ○ Faux

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Quelle extension officielle est recommandée pour travailler avec Python dans VSCode ?

- Python (par Microsoft)
- Code Runner
- GitLens

Question 3

Quel avantage majeur offre VSCode par rapport à un IDE plus complet comme PyCharm ?

- Plus de fonctionnalités intégrées
- Une meilleure intégration native avec Python
- Légèreté et rapidité d'utilisation

Question 4

Pourquoi est-il conseillé d'utiliser un environnement virtuel (venv) dans VSCode ?

- Pour installer plus facilement VSCode
- Pour améliorer la performance de VSCode
- Pour isoler les dépendances de chaque projet

Question 5

Quel est l'intérêt de ne pas surcharger VSCode avec trop d'extensions ?

- Pour éviter les conflits de versions
- Pour conserver des performances optimales
- Pour empêcher l'installation de packages

IV. Essentiel

Dans ce cours, nous avons exploré deux des outils les plus populaires pour programmer en Python, à savoir PyCharm et VSCode. Nous avons vu que PyCharm est un IDE complet et puissant, conçu spécifiquement pour Python, offrant des fonctionnalités avancées comme l'auto-complétion intelligente, le débogage graphique et l'intégration des frameworks Python. En parallèle, nous avons découvert VSCode, un éditeur de code plus léger et flexible, personnalisable grâce à un large choix d'extensions.

Nous avons également appris comment configurer un environnement de développement efficace en utilisant les environnements virtuels (venv). Cela permet d'isoler les dépendances de chaque projet et d'éviter les conflits de versions. L'installation et la gestion des packages Python ont été abordées, à la fois via l'interface graphique et le terminal, pour PyCharm comme pour VSCode.

Grâce à ces connaissances, vous avez désormais toutes les clés pour choisir l'outil qui correspond le mieux à vos besoins, que vous privilégiez la puissance et l'intégration de PyCharm ou la légèreté et la flexibilité de VSCode. L'essentiel est de se sentir à l'aise avec son environnement de travail, pour pouvoir se concentrer pleinement sur l'apprentissage et la pratique du langage Python !

Tous droits réservés à STUDI - Reproduction interdite

V. Auto-évaluation

A. Exercice

Mise en place d'un environnement Python et premiers exercices

Vous travaillez sur un projet de développement en Python et devez configurer un environnement de travail propre et fonctionnel. Ce projet vise à apprendre à créer un environnement virtuel, installer des packages essentiels et les utiliser dans un programme Python.

Question 1

[solution n°3 p.15]

Création d'un environnement virtuel et installation des packages essentiels

Crée un environnement virtuel dans l'IDE de ton choix et installe les packages Request et Pandas. Le nom du projet doit-être `my_python_project`. Puis à l'aide de la commande `pip3 list` vérifie que les packages ont bien été installés.

Question 2

[solution n°4 p.15]

Installer NumPy et l'utiliser dans un programme Python.

Dans votre projet `my_python_project` du premier exercice Installer NumPy et exécutez votre premier script avec le code suivant :

```python
import numpy as np
# Création d'un tableau NumPy
tableau = np.array([2, 4, 6, 8, 10])

# Calcul de la moyenne et de la somme
moyenne = np.mean(tableau)
somme = np.sum(tableau)

print("Tableau NumPy :", tableau)
print("Moyenne :", moyenne)
print("Somme :", somme)
```

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

studi

Exercice p. 7 Solution n°1

Question 1

PyCharm est développé par Microsoft.

- ○ Vrai
- ◉ Faux

PyCharm est développé par JetBrains, une entreprise spécialisée dans les outils de développement.

Question 2

Quelle édition de PyCharm propose des fonctionnalités avancées comme le support des frameworks web (Django, Flask) ?

- ○ PyCharm Community
- ◉ PyCharm Professional

PyCharm Professional propose des fonctionnalités avancées comme le support des frameworks web tels que Django et Flask.

Question 3

Quel est l'avantage principal d'utiliser un environnement virtuel (venv) dans PyCharm ?

- ○ Améliorer les performances du système
- ◉ Isoler les dépendances de chaque projet
- ○ Rendre le code Python plus rapide

Un environnement virtuel permet d'isoler les dépendances propres à chaque projet, évitant ainsi les conflits entre bibliothèques.

Question 4

Pourquoi est-il recommandé d'utiliser un environnement virtuel (venv) dans PyCharm ?

- ○ Pour améliorer les performances du processeur
- ◉ Pour isoler les dépendances de chaque projet et éviter les conflits entre bibliothèques
- ○ Pour installer automatiquement toutes les bibliothèques disponibles

L'utilisation d'un environnement virtuel permet d'avoir un espace de travail indépendant pour chaque projet, empêchant les conflits entre différentes versions de bibliothèques et facilitant la gestion des dépendances.

Question 5

Lors de la création d'un projet dans PyCharm, quelle étape ne fait pas partie du processus décrit dans le cours ?

- ○ Sélectionner la version de Python
- ◉ Choisir un thème d'interface
- ○ Donner un nom au projet

Tous droits réservés à STUDI - Reproduction interdite

Choisir un thème d'interface est proposé lors du premier lancement de PyCharm, pas nécessairement lors de la création d'un nouveau projet.

Exercice p. 10 **Solution n°2**

Question 1

VSCode est développé par JetBrains.

- ○ Vrai
- ◉ Faux

VSCode est développé par Microsoft et non par JetBrains.

Question 2

Quelle extension officielle est recommandée pour travailler avec Python dans VSCode ?

- ◉ Python (par Microsoft)
- ○ Code Runner
- ○ GitLens

L'extension officielle Python (par Microsoft) est recommandée pour bénéficier de l'auto-complétion avancée, du débogage intégré, et de la gestion des environnements virtuels.

Question 3

Quel avantage majeur offre VSCode par rapport à un IDE plus complet comme PyCharm ?

- ○ Plus de fonctionnalités intégrées
- ○ Une meilleure intégration native avec Python
- ◉ Légèreté et rapidité d'utilisation

VSCode est apprécié principalement pour sa légèreté, sa rapidité et sa flexibilité.

Question 4

Pourquoi est-il conseillé d'utiliser un environnement virtuel (venv) dans VSCode ?

- ○ Pour installer plus facilement VSCode
- ○ Pour améliorer la performance de VSCode
- ◉ Pour isoler les dépendances de chaque projet

Un environnement virtuel permet d'isoler les dépendances de chaque projet et d'éviter les conflits entre bibliothèques.

Question 5

Quel est l'intérêt de ne pas surcharger VSCode avec trop d'extensions ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Pour éviter les conflits de versions
- ◉ Pour conserver des performances optimales
- ○ Pour empêcher l'installation de packages

Installer trop d'extensions peut ralentir VSCode, il est donc préférable de ne choisir que celles qui sont vraiment nécessaires.

p. 12 Solution n°3

Voici la solution par lignes de commandes bien que vous puissiez obtenir le même résultat par l'interface graphique de l'IDE.

1. Créez un nouveau projet Python dans l'IDE de votre choix et nommez-le.

2. Créez un environnement virtuel (venv) :

```
python3 -m venv .venv
```

3. Activez l'environnement virtuel :

Sous Windows :

```
.venv\Scripts\activate
```

Sous MacOs/Linux

```
source .venv/bin/activate
```

4. Installez les packages voulus :

```
pip3 install requests pandas
```

Vérifiez que les packages sont bien installés avec :

```
pip3 list
```

p. 12 Solution n°4

1. Installez le packages NumPy :

```
pip3 install numpy
```

2. **Création du fichier Python**

Dans le projet **my_python_project**, créez un fichier nommé main**.py** et ajoutez le code suivant :

```
import numpy as np

# Création d'un tableau NumPy
tableau = np.array([2, 4, 6, 8, 10])

# Calcul de la moyenne et de la somme
moyenne = np.mean(tableau)
somme = np.sum(tableau)

print("Tableau NumPy :", tableau)
print("Moyenne :", moyenne)
print("Somme :", somme)
```

3. Exécutez le fichier **main.py** en ligne de commande :

Sous macOS/Linux :

```
python3 main.py
```

Tous droits réservés à STUDI - Reproduction interdite

Sous Windows :

```
python main.py
```

4. Vérification du résultat attendu

Après l'exécution, vous devriez voir l'affichage suivant dans la console :

```
Tableau NumPy : [ 2  4  6  8 10]
Moyenne : 6.0
Somme : 30
```


Tous droits réservés à STUDI - Reproduction interdite

Tous droits réservés à STUDI - Reproduction interdite